



Módulo

Trabalho de Conclusão da Capacitação

INSTITUIÇÃO EXECUTORA



Créditos

Autoria

Paulo Amaral

Revisão técnica

Nelson Almeida

Revisão pedagógica

Carla Dias

Helena Leme

Projeto gráfico e diagramação

Key Fernandes

Wellington Ribeiro

Instituição responsável

FIT - Instituto de Tecnologia

2026

Este material foi desenvolvido para fins educacionais. As interfaces, comandos e procedimentos apresentados correspondem às versões das ferramentas utilizadas durante sua elaboração e podem sofrer alterações.

Os nomes de produtos, softwares e marcas mencionados pertencem aos seus respectivos titulares e são utilizados exclusivamente para identificação e finalidade educacional.

INSTITUIÇÃO EXECUTORA



Sumário

Apresentação	4
.....	5
Unidade 1	5
Engenharia de Requisitos e Escopo de Projeto	5
Soluções que buscam por problemas.....	6
Levantamento de Requisitos.....	6
Delimitação de Escopo: Saber o que deixar de fora.....	8
Critérios de Sucesso e Testabilidade	9
Desenvolvendo a Arquitetura a partir da identificação da Dor	10
Unidade 2	11
Prototipagem Rápida e Prova de Conceito	11
Metodologia de Prototipagem e Validação Rápida (PoC)	12
O Risco da Integração Prematura	12
As Provas de Conceito (PoCs).....	13
Isolamento de Variáveis Críticas	13
O Ciclo Ágil: Falhando Rápido para Aprender Rápido	14
Unidade 3	15
Estruturação de Repositório e Roteirização	15
Boas Práticas de Documentação Técnica e Reprodutibilidade	16
Dependência de configurações locais	16
O Repositório como Produto: A Anatomia de um README	17
Reprodutibilidade: O Teste em Ambiente Limpo.....	18
Documentação Visual: A Integração do Esquemático	19
Da Teoria à Prática	20
Storytelling Técnico e Estruturação de Pitches	21
Comunicação entre o ambiente técnico e o ambiente de decisão	21
O Mapeamento de Valor: Foco no Problema Industrial	22
A Tradução do Jargão Técnico	23
Estrutura de um Pitch Técnico	24
Da Teoria à Prática: Síntese para a Apresentação.....	25
Unidade 4	27
Integração Final e Defesa do Projeto	27
Integração de Sistemas e Critérios de Qualidade de Entrega	28
O Desafio da Integração Final	28
Estratégias de Integração: Evitando o "Big Bang"	29
A Integração entre Edge AI, Firmware e Hardware.....	30
Estabilidade e Operacionalidade Mínima (Critérios de Aceite).....	30
Da Teoria à Prática - A Ponte para a Defesa do Projeto	32
Referências.....	33



Apresentação

Mais do que apenas aplicar conhecimentos técnicos em uma solução, o desenvolvimento de um projeto requer o planejamento e acompanhamento de suas etapas. Além de executar, aqueles que trabalham em um projeto precisam desenvolver habilidades de comunicação e uma vez que comumente é preciso tratar com clientes apresentando o que foi feito até então e com fornecedores para explicar quais suas demandas para aquele momento. A documentação é vital para que outros membros da equipe ou então futuros colaboradores se integrem em um nível técnico e consigam reproduzir ou dar prosseguimento ao desenvolvimento.

Esse material busca apresentar ferramentas utilizadas para o planejamento, desenvolvimento e acompanhamento de projeto com intuito de auxiliar desenvolver artefatos de entregas parciais e finais de projetos.

Bons estudos!



Unidade I

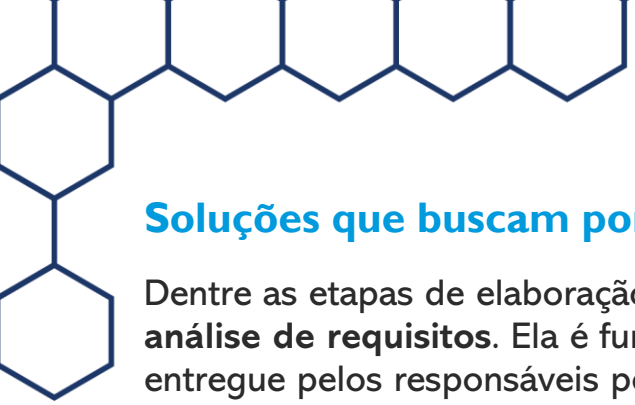
Engenharia de Requisitos e Escopo de Projeto

INSTITUIÇÃO EXECUTORA



COORDENADORA





Soluções que buscam por problemas

Dentre as etapas de elaboração de um projeto uma das mais importantes é a **análise de requisitos**. Ela é fundamental para que o escopo do que será entregue pelos responsáveis pela execução seja bem definido. Nessa fase, um risco enfrentado por desenvolvedores que se dedicaram ao estudo de uma determinada tecnologia é a tendência de priorizar sua aplicação em cenários onde ela não seria a melhor escolha.

Ferramentas como SBCs (single-board computers), ou computadores de placa única, avançados que suportam aplicações com redes neurais, apesar de sua versatilidade, trazem também alto custo monetário e de manutenção, além de adicionarem uma complexidade que muitas vezes não seria necessária para inúmeras situações. Durante a etapa de planejamento de uma solução, esses e muitos outros fatores precisam ser levados em consideração antes da estruturação do componente técnico de um projeto.

A lacuna de expectativa, ou *expectation gap*, é um conceito da engenharia de sistemas que representa justamente a distância entre o que o cliente realmente precisa e o que a equipe de desenvolvimento deseja construir por interesse técnico. Para reduzir o impacto dessa lacuna, a “dor”, ou seja, a necessidade do cliente, precisa ser identificada e declarada com clareza no início do projeto, aumentando as possibilidades de que a solução atenda ao problema proposto.

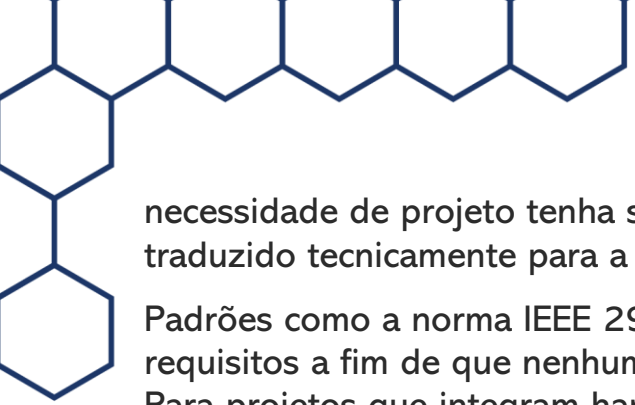
Por exemplo, em uma aplicação embarcada, as condições ambientais precisam ser levadas em consideração na definição do hardware. A temperatura de uma máquina, a exposição a poeira, o ruído elétrico, a distância entre pontos de alimentação e de rede podem causar mau funcionamento ou danos permanentes a determinados componentes.

Já a operacionalidade da solução influencia as escolhas relacionadas ao software. O balanço entre a precisão necessária e a latência ajuda a definir se um modelo complexo de *Deep Learning* (aprendizado profundo) é o mais adequado ou então uma simples lógica baseada em limiares.

Antes que a primeira linha de código seja escrita, a “dor”, ou a problemática a ser atendida, precisa ter sido bem estudada e definida com o foco da análise inicial nesse problema e não nas tecnologias que serão aplicadas.

Levantamento de Requisitos

A engenharia de sistemas é uma abordagem interdisciplinar e metodológica focada em como projetar, integrar e gerenciar sistemas complexos ao longo de todo seu ciclo de vida. Ela propõe diversas ferramentas para que, uma vez que a



necessidade de projeto tenha sido identificada e compreendida, o problema seja traduzido tecnicamente para a equipe de engenharia.

Padrões como a norma IEEE 29148 trazem a classificação minuciosa dos requisitos a fim de que nenhuma variável crítica do ambiente seja negligenciada. Para projetos que integram hardware e software, essa especificação é dividida em **requisitos funcionais** e **requisitos não-funcionais**.

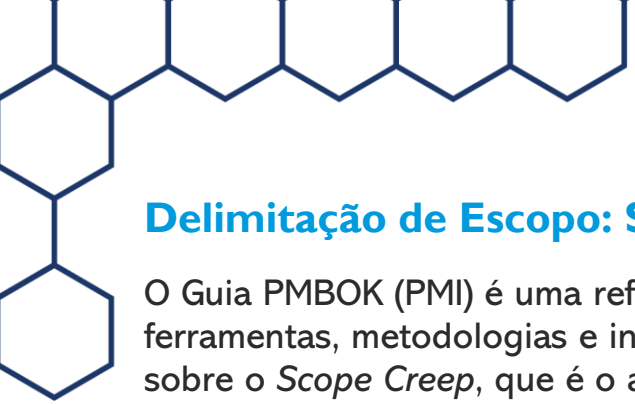
Os **requisitos funcionais** definem as ações, tarefas e comportamentos diretos que o sistema obrigatoriamente precisa executar para que o problema seja mitigado. Alguns exemplos de requisitos funcionais são: O nó de borda deve capturar imagens da esteira a cada 500 milissegundos ou O microcontrolador deve enviar um alerta via MQTT sempre que o sensor inercial detectar uma anomalia de vibração.

Enquanto os requisitos funcionais definem o que o sistema faz, os requisitos não funcionais definem quão bem ele faz isso e sob quais restrições. No contexto de sistemas embarcados no ambiente industrial, são essas restrições que definem a viabilidade da arquitetura. Por exemplo, relacionados ao desempenho e latência, perguntas como: “O modelo de Inteligência Artificial consegue processar a imagem e gerar a inferência em menos de 100 milissegundos para não comprometer a velocidade nominal da esteira?” precisam ser feitas.

Cuidados especiais precisam ser dados a depender do ambiente específico e a robustez requerida pela aplicação. É preciso saber se o dispositivo operará dentro de uma câmara frigorífica enfrentando condensação e variações térmicas drásticas ou colado à carcaça de um motor lidando com alto ruído eletromagnético e vibração constante.

Outra informação importante é relacionada à topologia e conectividade do sistema. O time precisa saber, por exemplo, se o Wi-Fi do chão de fábrica tem pontos cegos. Se sim, entender se o requisito exige que o sistema tenha capacidade de armazenar dados localmente até que a conexão seja restabelecida.

Ignorar ou não compreender esses requisitos não-funcionais é um grande vetor do fracasso em projetos de transições de tecnologia. É o que gera os "projetos de bancada", as provas de conceito que funcionam de maneira impecável em laboratório, com Wi-Fi estável e energia infinita, mas que não se sustentam em operação em ambientes mais exigentes como o industrial.



Delimitação de Escopo: Saber o que deixar de fora

O Guia PMBOK (PMI) é uma referência para gestores de projeto, compilando ferramentas, metodologias e informações relevantes nesse contexto. Ele alerta sobre o *Scope Creep*, que é o aumento descontrolado do escopo de projeto, muitas vezes fugindo de seu objetivo original. Esse fenômeno ocorre quando novas funcionalidades, quase sempre travestidas de “ideias rápidas e geniais”, são adicionadas gradativamente ao longo do desenvolvimento sem uma avaliação rigorosa do impacto no tempo, na estabilidade e na complexidade do sistema.

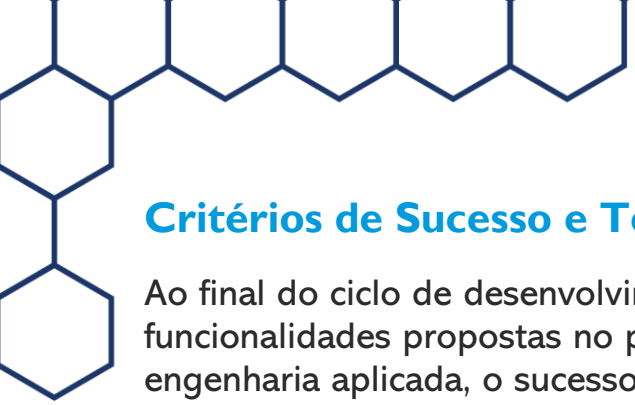
Para evitar que a equipe perca o foco e fracasse na entrega dentro do curto prazo estipulado para o projeto, é importante definir barreiras intransponíveis. Na engenharia de requisitos, conforme orienta Karl Wieggers por meio da declaração de *Limitations and Exclusions* (Limitações e Exclusões), documentar expressamente o que o sistema não se propusera a fazer é tão importante quanto definir o que ele fará.

No contexto de sistemas embarcados em ambiente industrial, estabelecer essa linha de corte determina até onde vai a autonomia do dispositivo de borda e onde começa a responsabilidade do operador humano ou do maquinário da fábrica.

Um exemplo pode ser visto nas seguintes definições de um projeto: O módulo de Visão Computacional identificará peças defeituosas na esteira em tempo real e publicará uma mensagem via protocolo MQTT para um painel gerencial, acendendo um LED de sinalização local, porém não será responsável por acionar válvulas solenoides para descartar a peça pneumáticamente, tampouco fará o corte de energia do motor da esteira.

Assumir a atuação física no ambiente industrial, algo deixado fora do escopo no exemplo acima, alteraria de forma drástica a lista de materiais, exigindo contadores, isolamento galvânico, protocolos de segurança rigorosos e aumentaria consideravelmente a complexidade do *firmware* e dos testes a serem executados.

Um escopo fechado, documentado e respeitado por todos os membros do grupo acaba sendo protetivo ao *squad* (esquadrão) de engenharia, garantindo que o desenvolvimento do protótipo permaneça viável, testável e com foco estrito na comprovação do valor central da solução.



Critérios de Sucesso e Testabilidade

Ao final do ciclo de desenvolvimento, é preciso comprovar que as funcionalidades propostas no projeto de fato foram entregues. Em ambientes de engenharia aplicada, o sucesso de uma solução não pode ser definido apenas pela compilação bem-sucedida de códigos-fonte, da ausência de falhas em um simulador ou até mesmo por um LED aceso em um teste de bancada. O sistema bem-sucedido é aquele capaz de resolver, de forma consistente, a dor industrial declarada no início do processo de planejamento.

O *Guia PMBOK* (PMI) defende a importância da presença nos projetos dos Indicadores Chave de Desempenho (KPIs) com métricas de sucesso que sejam quantificáveis, objetivas e, principalmente, testáveis. Sem a capacidade de mensurar o resultado e a estabilidade da sua solução, é difícil provar seu valor para o chão de fábrica.

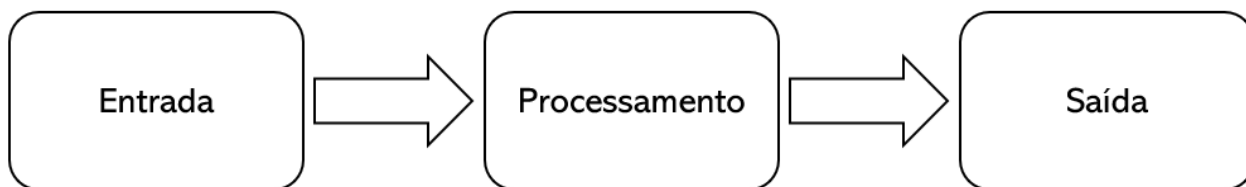
Os critérios de sucesso devem ser formulados como metas claras que o protótipo precisa atingir durante os testes de validação. Por exemplo, se o desafio é resolver a falta de visibilidade em uma esteira, a equipe não pode depender de um critério genérico. Os parâmetros precisam ser rigorosos e ancorados na realidade operacional. Definições subjetivas como: "A barreira óptica deve detectar de forma inteligente a passagem das caixas na esteira" são vagas, não especificam o que é inteligente e não pode ser testado matematicamente. Para esse caso, uma definição melhor seria: "O sensor não-intrusivo deve operar com uma acurácia mínima de 95% na contagem contínua de itens, suportando um fluxo de até 60 peças por minuto. Além disso, a lógica deve registrar autonomamente qualquer micro parada superior a 5 segundos, sem gerar falsos positivos."

Esses critérios definidos na documentação atuarão como a bússola para o desenvolvimento. Durante as fases de *debug* e ajustes, o que deve guiar a equipe deve ser sempre se a alteração feita contribui para que a métrica definida inicialmente seja cumprida.

Desenvolvendo a Arquitetura a partir da identificação da Dor

Com a dor compreendida, o escopo protegido de desvios e os requisitos funcionais, não-funcionais e métricas de sucesso devidamente documentados, o problema deixa de ser apenas uma ideia abstrata. Pode então ser elaborado o Diagrama de Arquitetura, um modelo visual e lógico que descreve tecnicamente a solução.

O conceito de Design Arquitetural é popular na Engenharia de Sistemas, sendo Roger S. Pressman um dos que abordam esse conceito em seus materiais escritos. Ele descreve, entre outras coisas, a arquitetura orientada ao fluxo de dados. Para sistemas embarcados, essa acaba sendo uma abordagem simples e eficiente.



Nesse contexto, as entradas dizem respeito a quais componentes (câmeras, acelerômetros, botões, chaves ópticas) capturarão os dados do ambiente físico.

O processamento é o nó computacional (como uma Raspberry Pi ou ESP32) que receberá os dados brutos e sua inteligência aplicada localmente (modelos de IA, inferência via Edge Impulse, cálculos lógicos de tempo e limiar).

Por fim, as saídas serão para onde o resultado do processamento será enviado, descrevendo se haverá atuação local (um display OLED, um LED de alarme, um *buzzer*) ou comunicação remota (transmissão de um *payload* MQTT via Wi-Fi para um painel gerencial).

O esforço investido na Engenharia de Requisitos aumenta as chances de um projeto ser bem-sucedido ao ajudar a definir e visualizar aspectos importantes do mesmo. Um documento de escopo bem redigido e um diagrama arquitetural claro são os contratos técnicos do projeto, garantindo que o time saiba exatamente o que precisa ser construído e sob quais métricas o resultado será julgado.



Unidade 2

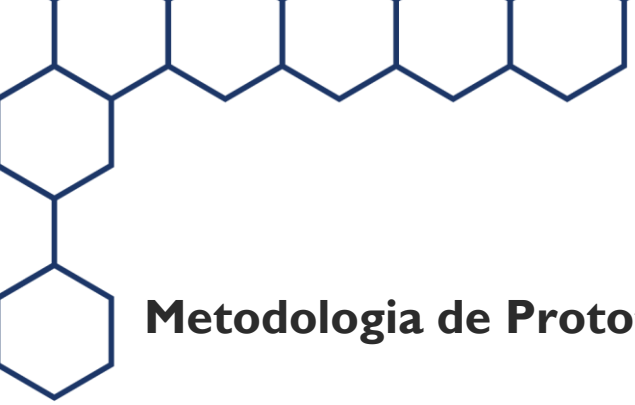
Prototipagem Rápida e Prova de Conceito

INSTITUIÇÃO EXECUTORA



COORDENADORA





Metodologia de Prototipagem e Validação Rápida (PoC)

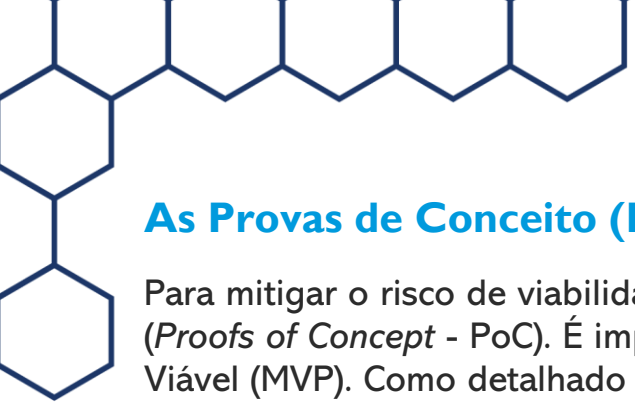
O Risco da Integração Prematura

Uma vez que o documento de requisitos esteja pronto e a arquitetura do projeto desenhada, pode parecer natural que o próximo passo direto seja a construção do sistema. A conexão de sensores, escrita do firmware, configuração da rede Wi-Fi e do broker MQTT e o desenho do painel de visualização, tudo ao mesmo tempo, em poucos dias e sem pensar muito bem na ordem em que cada uma dessas coisas é executada. Na engenharia de software, essa prática é conhecida como *Big Bang Integration* (Integração Big Bang), algo não muito aconselhável.

A integração de todas as peças tecnológicas logo no início tem grande probabilidade de ser acompanhada de falhas. Com muitos elementos conectados, a rastreabilidade de o que as causaram é muito mais difícil. Comportamentos erráticos similares podem ter origens distintas e ainda em subsistemas totalmente diferentes. O sistema pode travar porque ocorreu um mau contato no hardware, porque o roteador derrubou o pacote de rede, ou porque há um erro de lógica na máquina de estados.

A engenharia de produtos possui algumas ferramentas para evitar que esses erros ocorram, sempre considerando a gestão de riscos com olhar atento. Em seu livro *Inspired*, Marty Cagan destaca que o desenvolvimento de qualquer solução tecnológica enfrenta diversos riscos, sendo o mais crítico nesta fase inicial o Risco de Viabilidade (*Feasibility Risk*). Em termos práticos, é a equipe de engenharia avaliando com sinceridade se realmente é capaz construir a inteligência da solução proposta com a tecnologia, o tempo e o conhecimento que tem hoje.

Assim, o objetivo da etapa inicial de um projeto deixa de ser o desenvolvimento do produto completo e passa a ser o de mitigar o risco técnico. Antes de pensar em integração, o foco deve ser estritamente provar que a ideia central é tecnicamente possível antes de desperdiçar horas valiosas de desenvolvimento nas partes periféricas do sistema.



As Provas de Conceito (PoCs)

Para mitigar o risco de viabilidade técnica, são feitas as Provas de Conceito (*Proofs of Concept* - PoC). É importante não confundir PoC com Produto Mínimo Viável (MVP). Como detalhado por Eric Ries em *A Startup Enxuta*, o MVP é construído para testar hipóteses de aderência junto ao usuário final com o fim de testar aspectos funcionais da solução final como, por exemplo, se operador de uma esteira vai realmente olhar para o painel de dados construído pela equipe durante seu turno de trabalho. Já a PoC é uma experimentação interna da equipe de engenharia focada estritamente em validar as maiores fragilidades tecnológicas como, por exemplo, se a lógica desenvolvida no microcontrolador consegue processar o bloqueio da luz e calcular a taxa de produção sem causar travamentos no processador.

Conceitualmente o resultado da PoC é rudimentar e frequentemente descartável. Ela não é feita para ser escalável ou esteticamente agradável. Para montar uma PoC, não é preciso uma placa de circuito impresso definitiva, uma estrutura de banco de dados na nuvem, ou então um código com tratamento perfeito de exceções. Fios emaranhados em uma *protoboard* ou um algoritmo com uma lógica mais “crua” rodando isolada em um simulador são cenários perfeitamente aceitáveis.

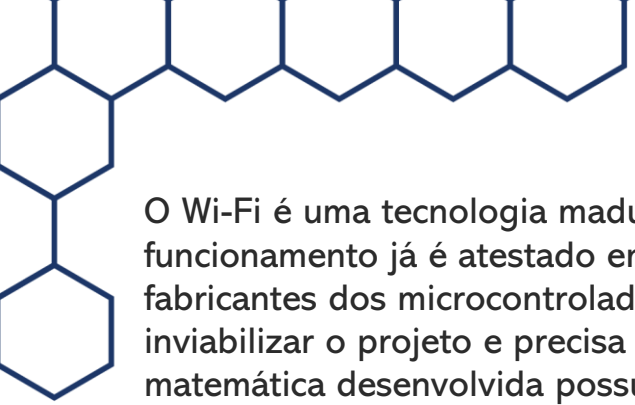
O único objetivo de uma PoC é forçar o núcleo da ideia arquitetural a responder a uma pergunta técnica binária e implacável, se a premissa fundamental funciona ou se não, essa abordagem de hardware/software é incapaz de entregar a métrica necessária. O custo de descobrir um não para a capacidade técnica da solução é consideravelmente mais baixo em estágio de PoC, pois o time gastou apenas alguns dias testando o limite do sensor, e não semanas programando telas e servidores para um projeto que já não iria funcionar por conta de uma premissa tecnológica equivocada.

Isolamento de Variáveis Críticas

Uma vez que a PoC não contempla o sistema completo, a decisão de o que deve ser construído e testado primariamente passa por entender, ao observar o Diagrama de Arquitetura, qual seria a variável técnica mais arriscada.

Em seu livro *Inspired*, ao detalhar a construção de Protótipos de Viabilidade (*Feasibility Prototypes*), Marty Cagan orienta que os desenvolvedores devam focar no algoritmo central, na precisão do sensor ou nas limitações de performance, ignorando sumariamente todos os componentes periféricos.

Em um sistema que utiliza uma célula de carga para aferir o peso aliado à conectividade Wi-Fi para sinalizar à logística quando uma caixa de peças está vazia, a conexão de um microcontrolador ao Wi-Fi não é a maior fragilidade.



O Wi-Fi é uma tecnologia madura, amplamente documentada e cujo funcionamento já é atestado em diferentes exemplos providos pelos próprios fabricantes dos microcontroladores. O verdadeiro risco técnico, o que pode inviabilizar o projeto e precisa ser validado, é se a célula de carga e a lógica matemática desenvolvida possuem precisão suficiente para detectar a retirada de um único parafuso da caixa, sem gerar flutuações falsas na leitura.

Nesse caso, é exatamente isso que a PoC deveria isolar. Durante essa validação, a equipe deve relevar temporariamente o display OLED, o botão de *reset* manual, a formatação do painel gerencial ou o banco de dados na nuvem. Todo o esforço de codificação e hardware deve ser direcionado para garantir que a leitura da variação de peso funcione perfeitamente, isolando a variável crítica e testando-a à exaustão.

O Ciclo Ágil: Falhando Rápido para Aprender Rápido

Uma vez que a variável crítica foi isolada na Prova de Conceito, os desenvolvedores devem colocá-la à prova imediatamente. No desenvolvimento de hardwares e sistemas embarcados, as primeiras iterações podem não se comportar conforme o planejado no papel. Por conta disso, a cultura ágil defende um lema contraintuitivo para a engenharia tradicional: falhe rápido para aprender rápido.

Em *Lean Startup*, Eric Ries popularizou o ciclo de *feedback* ininterrupto conhecido como Construir, Medir e Aprender. Em um contexto de sistemas embarcados, o Construir seria escrever a lógica principal e montar o circuito mínimo, físico em protoboards ou em simuladores. O Medir seria injetar dados no sistema e analisar as saídas verificando, por exemplo, se um sensor inercial disparou no tempo certo ou a barreira óptica calculou a taxa de OEE (Eficiência Global do Equipamento) corretamente quando a luz foi interrompida. E por fim, o Aprender é a avaliação dos resultados. O sistema pode ter gerado, por exemplo, falsos positivos devido ao ruído elétrico, indicando a necessidade de desenvolver um filtro matemático, como um *debounce*, antes de prosseguir.

Assim, um erro inicial não é um fracasso do projeto, mas sim um dado de calibração. Descobrir que a lógica condicional de um alarme térmico está falhando durante a segunda semana de projeto é um sucesso imenso. Isso confere à equipe tempo hábil para corrigir a rota, trocar o componente virtual ou refatorar o código. O verdadeiro desastre seria descobrir essa mesma falha estrutural apenas na véspera da entrega final, quando todo o sistema já está praticamente finalizado.



Unidade 3

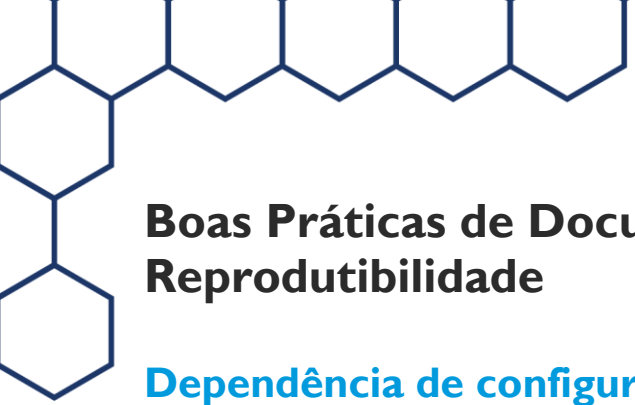
Estruturação de Repositório e Roteirização

INSTITUIÇÃO EXECUTORA



COORDENADORA





Boas Práticas de Documentação Técnica e Reprodutibilidade

Dependência de configurações locais

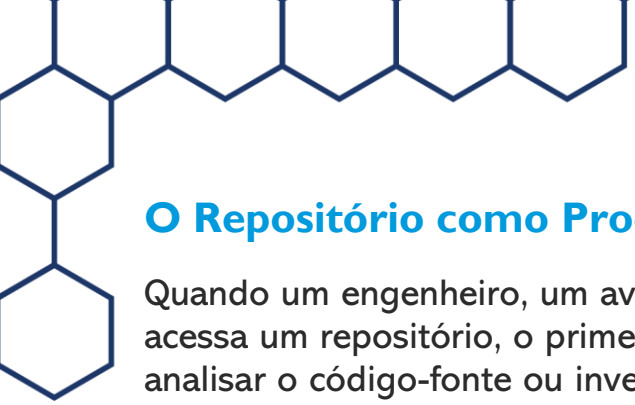
A frase “na minha máquina estava funcionando” representa uma situação recorrente na engenharia de software e no desenvolvimento de sistemas embarcados.

Existe uma interpretação comum de que o desenvolvimento de um projeto termina no exato momento em que o código compila sem erros ou quando a Prova de Conceito (PoC) roda com sucesso no *laptop* do autor. Na prática da engenharia, um código não documentado, sem controle de versão adequado e atrelado a configurações locais é, para todos os efeitos práticos, um código com baixa capacidade de reprodução. O valor de uma solução tecnológica também está relacionada à facilidade com que outra pessoa ou um sistema automatizado consegue executá-la.

A execução não deve depender de intervenções manuais não documentadas nem de conhecimento concentrado em uma única pessoa. Se o sistema exige que alguém descubra qual versão de uma biblioteca precisa ser instalada ou em qual porta COM o microcontrolador deve estar conectado, ele se torna uma solução com reprodução dificultada.

Uma prática observada em algumas equipes no início de seus projetos é tratar o repositório de versionamento, como GitHub ou GitLab, apenas como um depósito de arquivos ou como cópia de segurança.

Em contextos profissionais, o repositório funciona como fonte central de informações sobre a solução. Ele é um canal de comunicação entre a equipe responsável, as pessoas que analisarão o código e os fluxos automatizados. Se a arquitetura pensada organiza o projeto, e a PoC testada demonstra seu funcionamento, o repositório estruturado é o elemento que garante que essa solução sobreviva e seja útil fora da bancada original.



O Repositório como Produto: A Anatomia de um README

Quando um engenheiro, um avaliador ou até mesmo um sistema automatizado acessa um repositório, o primeiro arquivo lido é o README.md. Não é praticoa analisar o código-fonte ou investigar os arquivos de configuração antes de entender o que aquele amontoado de arquivos faz. O README funciona como a página inicial, a vitrine e o manual de instrução primário da sua solução.

Uma dificuldade frequente é escrever a documentação presumindo que o leitor possui o mesmo contexto dos autores. Na estruturação de repositórios, convém escrever assumindo que quem acessou o link ainda não sabe qual problema está sendo tratado nem como a arquitetura foi organizada.

Para que o repositório deixe de ser apenas um depósito de arquivos e possa apoiar a reprodução do projeto, a anatomia de um bom README.md deve responder de forma clara a quatro perguntas estruturais:

1. O que é este projeto? (Visão Geral):

Um parágrafo direto detalhando qual foi o problema industrial tratado e como o sistema busca tratá-lo, consolidando o escopo e a proposta técnica previamente definidos.

2. O que compõe a solução? (Stack e Lista de Materiais):

Quais placas (ex: ESP32), atuadores e sensores formam a barreira física? Quais linguagens de programação e *frameworks* formam a barreira lógica?

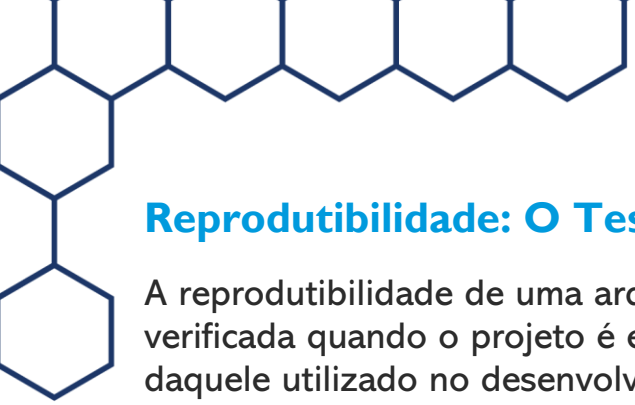
3. O que eu preciso ter antes de começar? (Pré-requisitos):

Quais softwares de base (ex: Docker, extensões específicas de IDE, interpretadores) e drivers o usuário precisa ter na máquina antes de tentar rodar o projeto?

4. Como eu rodo isso na prática? (Instruções de Setup):

Um roteiro passo a passo contendo os comandos necessários para clonar o projeto, instalar as bibliotecas, compilar o firmware e executar os testes de validação.

Se o README estiver bem redigido, uma pessoa com conhecimento técnico compatível conseguirá compreender a lógica do projeto e reproduzir suas funções principais com base nas informações registradas, sem depender de explicações adicionais ou informações externas.



Reprodutibilidade: O Teste em Ambiente Limpo

A reprodutibilidade de uma arquitetura de software embarcado ou IoT pode ser verificada quando o projeto é executado em um ambiente limpo, diferente daquele utilizado no desenvolvimento. Para isso, o repositório pode ser clonado em um computador com as ferramentas básicas instaladas ou em um ambiente de integração contínua na nuvem, como um executor (runner) do GitHub Actions. Esse teste permite verificar se as instruções, dependências e configurações registradas são suficientes para preparar e executar as funções principais da solução.

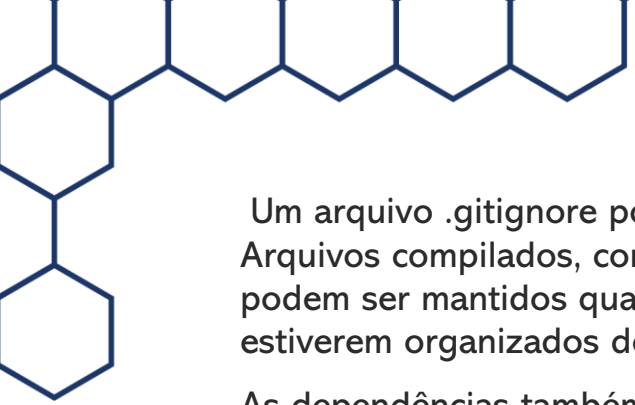
Processos manuais e ambientes de desenvolvimento dependentes de configurações específicas dificultam a reprodução e a manutenção do projeto. Sempre que possível, as etapas de preparação, compilação e execução devem ser automatizadas, e o repositório deve reunir os arquivos, as dependências e as instruções necessárias para recriar o ambiente de forma consistente.

Para favorecer a reprodutibilidade, devem ser evitados três problemas recorrentes:

Diretórios absolutos: o código não deve buscar arquivos de configuração por meio de caminhos específicos do computador utilizado no desenvolvimento, como `C:\Users\Aluno\Desktop\Projeto\config.json`. Esse caminho pode não existir em outro ambiente, impedindo a execução. Sempre que possível, devem ser utilizados caminhos relativos, como `./config.json`, mantendo os arquivos necessários dentro da estrutura do repositório.

Credenciais incorporadas ao código: senhas de redes Wi-Fi, tokens de acesso a brokers MQTT e chaves de API não devem ser registradas em texto aberto no código-fonte, como em arquivos `.cpp` ou `.py`. Além de representar um risco de segurança, essa prática dificulta a configuração da solução em outro ambiente. Essas informações devem ser fornecidas por arquivos externos ou variáveis de ambiente. Arquivos como `.env` não devem ser versionados, mas um arquivo de exemplo, como `.env.example`, pode indicar quais variáveis precisam ser configuradas, sem expor dados sensíveis.

Arquivos desnecessários e dependências não registradas: arquivos temporários, caches, executáveis e configurações específicas do ambiente local devem ser excluídos do versionamento quando não forem necessários para compreender, executar ou distribuir a solução.



Um arquivo `.gitignore` pode ser utilizado para controlar essas exclusões. Arquivos compilados, como `.bin` e `.exe`, e configurações da pasta `.vscode` podem ser mantidos quando tiverem uma finalidade documentada e estiverem organizados de forma adequada.

As dependências também precisam ser registradas de acordo com a linguagem e o ambiente utilizados. Um projeto em Python pode utilizar um arquivo `requirements.txt`. Em projetos para ESP32, as bibliotecas, os componentes e o processo de compilação devem ser registrados nos arquivos compatíveis com o ambiente adotado, como os arquivos de configuração do ESP-IDF ou, quando aplicável, o `platformio.ini`.

Contêineres Docker podem ser utilizados quando fizerem parte da solução, mas não são obrigatórios para garantir a reprodução.

A reprodutibilidade depende de instruções claras para instalar as dependências, configurar o ambiente, compilar o código e executar as funções principais. Uma pessoa com conhecimento técnico compatível ou um processo automatizado de integração e entrega contínuas (CI/CD) deve conseguir realizar essas etapas com base nos comandos e nas orientações descritos no README, sem depender de ajustes manuais não documentados.

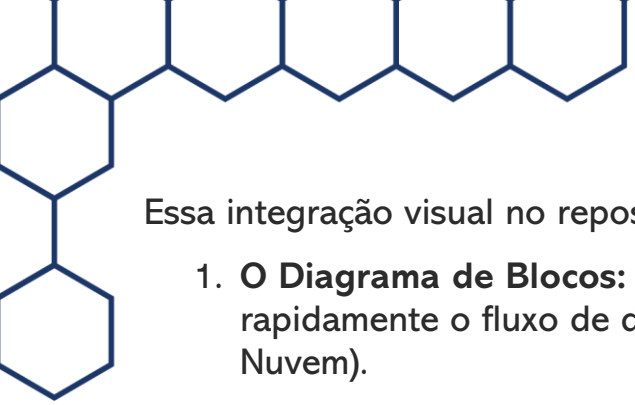
Documentação Visual: A Integração do Esquemático

Em sistemas envolvendo hardware e software, como projetos de IoT, robótica e *Edge AI*, disponibilizar um código-fonte limpo e com dependências gerenciadas resolve apenas parte do problema da reprodutibilidade.

Como o software interage com componentes físicos, o código é insuficiente se quem for executá-lo não souber como montar o hardware que lhe dá suporte.

A documentação da arquitetura por meio de modelos visuais é uma prática importante na engenharia de software. A representação gráfica do sistema é uma forma de transferir conhecimento técnico e de contribuir para que a estrutura física projetada seja coerente com a lógica implementada no código.

Em um repositório estruturado, a documentação visual deve acompanhar o desenvolvimento lógico. Se o *firmware* está programado para ler pulsos de um sensor infravermelho no pino analógico 34 e acionar um relé no pino 2, essa topologia não pode existir apenas como conhecimento da equipe. Ela precisa estar formalmente documentada no Git, frequentemente armazenada em uma pasta dedicada, como `/docs` ou `/assets`).



Essa integração visual no repositório se traduz em duas entregas principais:

1. **O Diagrama de Blocos:** Ele é inserido no README para explicar rapidamente o fluxo de dados em alto nível (ex: Sensor -> ESP32 -> Nuvem).
2. **O Esquemático Elétrico (Físico):** Uma representação visual ou diagrama de pinagem da *protoboard* (que pode ser gerado em ferramentas de simulação ou esquemáticos eletrônicos).

Se o código for clonado por outra pessoa, ela deve conseguir consultar a documentação visual, conectar os fios nos mesmos pinos descritos no diagrama, compilar o código e verificar o sistema nas condições registradas. A coerência entre o código submetido e o diagrama anexado é uma evidência relevante de reprodutibilidade em sistemas embarcados.

Da Teoria à Prática

Após a validação do risco técnico e da viabilidade da solução por meio da Prova de Conceito (PoC), o passo seguinte é consolidar o repositório do projeto de modo que ele possa ser compreendido, executado e avaliado em outro ambiente, sem depender da configuração local de seus autores.

A consolidação do repositório requer a organização dos arquivos, das dependências, das configurações e das instruções do projeto. O arquivo `.gitignore` deve ser configurado para evitar o versionamento de credenciais, arquivos temporários e elementos específicos do ambiente local que não contribuam para a compreensão, a execução ou a distribuição da solução. Artefatos compilados e configurações compartilháveis podem ser mantidos quando possuírem uma finalidade documentada e estiverem organizados adequadamente.

A análise técnica deve considerar não apenas o funcionamento do circuito, mas também a clareza e a consistência do repositório. O arquivo `README.md` deve apresentar o problema, os pré-requisitos, as instruções de configuração e os comandos necessários para executar o projeto. A documentação visual, o diagrama de blocos e, quando aplicável, o esquemático de montagem deve corresponder à lógica implementada no firmware.

A reprodutibilidade pode ser verificada por meio de um teste em ambiente limpo. Esse procedimento permite avaliar se a arquitetura, o código, as dependências e as instruções registradas são suficientes para preparar e executar as funções principais da solução sem depender de ajustes manuais ou conhecimentos não documentados.



Storytelling Técnico e Estruturação de Pitches

Comunicação entre o ambiente técnico e o ambiente de decisão

Na engenharia de sistemas e no desenvolvimento de hardware, existe a percepção de que um projeto tecnicamente consistente comunica seu valor por si só. Entretanto, a qualidade técnica não elimina a necessidade de explicar com clareza o problema, a solução desenvolvida e os resultados observados.

É comum que equipes dediquem meses à construção de lógicas em microcontroladores, à otimização de modelos de Edge AI e à redução da latência de comunicação, sem que isso seja suficiente para que a solução seja compreendida ou priorizada em contextos de decisão. O motivo pode não estar no código. Em muitos casos, a dificuldade está na tradução da solução técnica para uma linguagem compatível com o contexto de negócio. A distância entre a bancada de desenvolvimento e o ambiente de decisão pode estar associada a falhas de comunicação.

Em obras voltadas à comunicação, ao propósito e à liderança, o Círculo Dourado (Golden Circle) é apresentado como uma estrutura que organiza a mensagem a partir de três questões: por que, como e o que. Aplicado ao desenvolvimento de soluções IoT e sistemas industriais, esse princípio indica que uma apresentação pode começar pelo problema e pela finalidade da solução antes de detalhar seus componentes e recursos técnicos.

Em uma apresentação técnica, por exemplo, a descrição pode começar pela topologia do sistema:

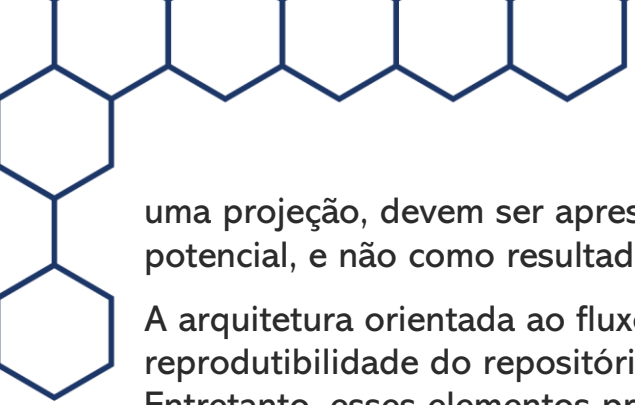
Desenvolvemos um nó de processamento na borda utilizando um ESP32 programado em C++ para analisar dados inerciais em tempo real.

Essa frase apresenta informações relevantes sobre a arquitetura, mas pode não esclarecer imediatamente qual problema está sendo tratado e qual função a solução desempenha.

O storytelling técnico, ou narrativa técnica, propõe uma organização diferente. Antes de mostrar a placa de circuito impresso, a câmera ou o painel de dados, a apresentação deve contextualizar a situação observada, por exemplo:

“A fábrica apresenta perdas recorrentes porque não identifica com antecedência sinais de falha nos motores. A solução proposta busca apoiar a detecção dessas condições com maior rapidez”.

Dados sobre perdas, falhas ou desperdícios podem ser utilizados quando forem sustentados por registros ou fontes adequadas. Quando representarem apenas



uma projeção, devem ser apresentados como impacto estimado ou benefício potencial, e não como resultado comprovado.

A arquitetura orientada ao fluxo de dados, a Prova de Conceito (PoC) e a reprodutibilidade do repositório fornecem sustentação técnica à solução. Entretanto, esses elementos precisam ser relacionados ao problema, às condições de teste, aos resultados observados e às limitações identificadas. A tecnologia constitui o meio utilizado para desenvolver a proposta, enquanto sua relevância depende da adequação ao problema e da qualidade das evidências apresentadas.

O Mapeamento de Valor: Foco no Problema Industrial

A transição entre o desenvolvimento técnico e a comunicação da solução exige que a equipe aprenda a mapear o valor de sua entrega. Uma vez que o "Porquê" foi estabelecido, é necessário conectar os requisitos funcionais e não-funcionais já definidos aos benefícios de negócio que eles podem gerar na prática.

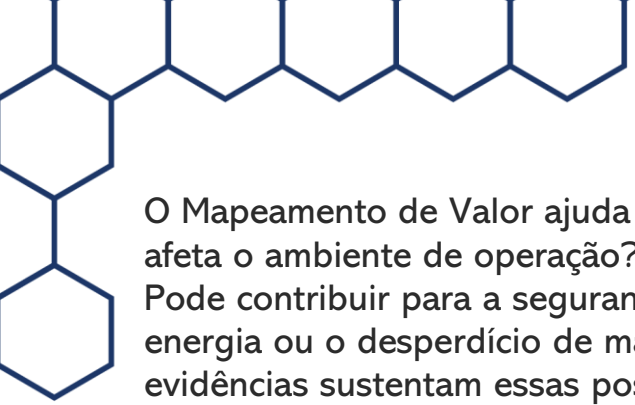
Em abordagens de design de proposta de valor, o Mapa de Valor é apresentado como um modelo visual que relaciona as funcionalidades de uma solução às dificuldades que ela pode reduzir e aos benefícios que pode proporcionar. Nesse contexto, aparecem os conceitos de aliviadores de dificuldades (Pain Relievers) e criadores de benefícios (Gain Creators), associados às necessidades e às atividades do público envolvido.

Para a engenharia, isso significa analisar o projeto não apenas pelos componentes de hardware ou pelos blocos de código utilizados, mas também pela contribuição da solução para o problema identificado. Gestores, responsáveis pelo processo e outros públicos precisam compreender o que os recursos técnicos permitem realizar e em quais condições essa contribuição foi observada.

Considere o seguinte cenário de inspeção visual:

- **A visão do desenvolvedor, focada em funcionalidade:** “Nós instalamos uma câmera de 5 megapixels rodando um modelo de rede neural convolucional (YoloV8) processado na borda a 30 *frames* por segundo, com comunicação via MQTT.”
- **A visão de Negócio, focada em mapear valor:** “Nós desenvolvemos um sistema de inspeção automatizada que substitui a amostragem visual manual. Isso zera o envio de peças com defeito de usinagem para o cliente final e evita multas por devolução de lote.”

A câmera, a rede neural e o protocolo MQTT são os recursos tecnológicos que tornam o alívio da dor possível.



O Mapeamento de Valor ajuda a responder a questões como: como o protótipo afeta o ambiente de operação? Pode melhorar a rastreabilidade das ocorrências? Pode contribuir para a segurança ergonômica? Pode reduzir o consumo de energia ou o desperdício de matéria-prima? Que resultados observados ou evidências sustentam essas possibilidades?

Quando o pitch, ou a apresentação, não relaciona claramente os recursos técnicos ao problema e às evidências disponíveis, a solução pode parecer mais complexa ou de custo pouco justificado, mesmo que o código esteja funcional e a arquitetura seja tecnicamente consistente.

A Tradução do Jargão Técnico

Um dos maiores desafios ao apresentar um projeto é adaptar a linguagem ao repertório do público. Quando profissionais passam longos períodos envolvidos com documentações, esquemáticos e linhas de código, termos como Edge Computing, broker MQTT, latência e barramento I2C tornam-se parte de seu vocabulário habitual. Para gestores, responsáveis pelos processos ou outras pessoas que não participaram do desenvolvimento, esses termos podem exigir uma explicação simples e objetiva. Quando a linguagem não é compreendida, o entendimento da apresentação pode ser prejudicado.

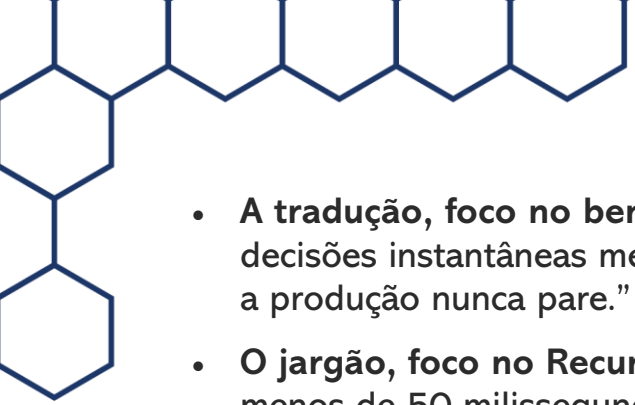
Em abordagens de storytelling inspiradas em apresentações de Steve Jobs, a empatia comunicativa é destacada como uma prática importante. Em vez de concentrar a apresentação em megahertz, barramentos de memória ou linhas de código, conceitos técnicos complexos podem ser relacionados a analogias simples, funções e possíveis benefícios.

Para a equipe de desenvolvimento, isso significa aprender a converter os recursos (*features*) do sistema em benefícios (*benefits*).

Detalhes como a frequência de clock do microcontrolador podem ser necessários para justificar uma decisão técnica, mas outros públicos também precisam compreender como a solução se relaciona à disponibilidade do equipamento, ao OEE (Eficiência Global do Equipamento), aos custos operacionais ou a outros aspectos pertinentes. Esses impactos devem ser mencionados somente quando houver dados ou evidências que os sustentem.

Veja como a tradução do jargão transforma completamente a percepção de valor:

- **O jargão, foco no recurso:** "O nosso nó sensor possui processamento local (Edge AI) e não depende de requisições HTTPS para a nuvem."

- 
- **A tradução, foco no benefício:** “O nosso sistema funciona e toma decisões instantâneas mesmo se a internet da fábrica cair, garantindo que a produção nunca pare.”
 - **O jargão, foco no Recurso:** “O sistema publica um *payload* MQTT em menos de 50 milissegundos.”
 - **A tradução, foco no benefício:** “O sistema alerta o operador em tempo real, segundos antes da quebra catastrófica do motor, salvando milhares de reais em manutenção corretiva.”

Se for absolutamente necessário explicar um detalhe técnico da arquitetura para justificar a inovação, podem ser utilizadas analogias. Uma rede neural, por exemplo, pode ser descrita como um sistema treinado com exemplos para reconhecer determinados padrões, semelhante a um inspetor que aprendeu a identificar características esperadas em uma peça. Essa comparação deve considerar que os resultados dependem da qualidade dos dados, do treinamento e das condições de uso. A simplificação da linguagem deve facilitar a compreensão sem eliminar a precisão técnica ou criar promessas não sustentadas.

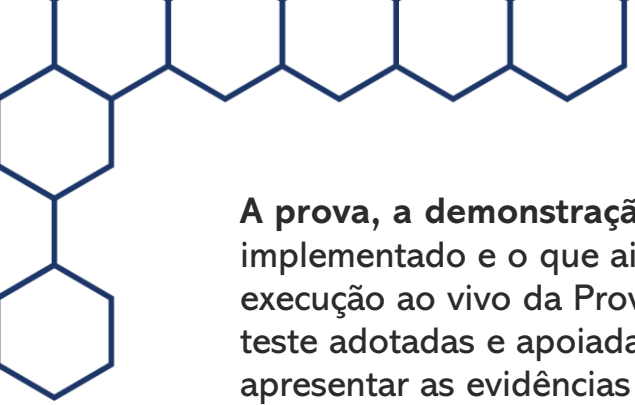
Estrutura de um Pitch Técnico

Com a “dor”, ou o problema, mapeado e o jargão traduzido, o desafio seguinte é organizar a narrativa em um roteiro coeso e compatível com o tempo disponível. Um pitch técnico (uma apresentação breve e estruturada) não precisa abordar todas as funcionalidades do sistema. Seu objetivo é conduzir o público, dentro de uma jornada lógica, do problema à solução, às evidências produzidas e às possibilidades de continuidade.

Para favorecer a compreensão por diferentes públicos, a apresentação pode ser organizada em quatro partes sequenciais:

O gancho, a dor: Comece pelo cenário, não pelo código. “Qual é o problema real no ambiente de aplicação?” e, mais importante, “Quanto ele custa atualmente em tempo, dinheiro, matéria-prima ou segurança?” Prenda a atenção da audiência demonstrando que a equipe compreende a raiz da dor operacional.

A solução, a proposta de valor: Apresente a arquitetura tecnológica. De forma clara e traduzida, sem abusar de jargões para o negócio, explique como o sistema *Edge/IoT* ataca diretamente o problema apresentado no gancho. Destaque as principais decisões técnicas e os critérios que justificaram a abordagem arquitetural escolhida.



A prova, a demonstração: Esta é a etapa de mostrar o que foi implementado e o que ainda permanece como possibilidade. O vídeo ou a execução ao vivo da Prova de Conceito (PoC), validada nas condições de teste adotadas e apoiada por documentação técnica reproduzível, ajuda a apresentar as evidências disponíveis. A demonstração visual é uma evidência de que o risco tecnológico foi mitigado e de que a solução funcionou nas condições testadas.

O futuro, o fechamento: Em vez de encerrar a apresentação apenas com a demonstração realizada, a equipe pode concluir projetando o impacto da solução. Uma possibilidade é indicar qual poderia ser o ganho operacional (OEE), ou a redução de custos (ROI), caso o protótipo fosse ampliado para o ambiente de operação.

Essa estrutura de quatro passos orienta a apresentação a reduzir pontos menos relevantes para o objetivo da apresentação. Ela indica que a equipe não apenas domina a implementação técnica, como detalhes do microcontrolador e funcionalidades disponíveis pelo *framework*, mas demonstra maturidade técnica e comercial para responder a problemas complexos em contextos reais de aplicação.

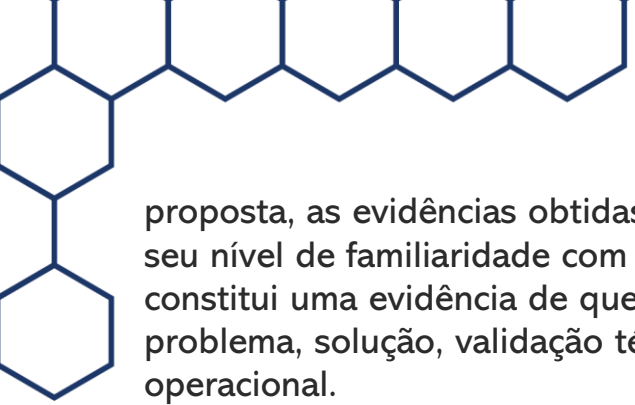
Da Teoria à Prática: Síntese para a Apresentação

A apresentação reúne os principais elementos desenvolvidos no projeto: a identificação da “dor” industrial, ou seja, do problema a ser tratado, sua tradução em requisitos técnicos e arquiteturais, a validação da Prova de Conceito (PoC) e a organização do repositório com foco em reprodutibilidade.

No entanto, a etapa final do projeto não se resume ao envio de um link do GitHub ou de um diagrama esquemático. Ela também envolve a apresentação pública da solução por meio de um pitch executivo, no qual a equipe comunica, de forma breve e estruturada, o problema tratado, a solução proposta, as evidências obtidas e os impactos esperados.

Na prática, a equipe terá apenas alguns minutos diante da banca avaliadora para sintetizar o desenvolvimento realizado em uma narrativa coesa e compatível com o tempo disponível. O desafio desta etapa é adequar a linguagem ao público, aplicar o mapeamento de valor e estruturar uma apresentação organizada em torno de quatro elementos: o problema identificado, a solução proposta, as evidências apresentadas pela Prova de Conceito (PoC) e as possibilidades futuras de aplicação ou ampliação da solução.

A apresentação tende a ser mais efetiva quando a banca avaliadora, ou outros públicos de decisão, compreende o problema tratado, o funcionamento da



proposta, as evidências obtidas e os impactos esperados, independentemente de seu nível de familiaridade com linguagens de programação ou eletrônica. Isso constitui uma evidência de que o trabalho foi desenvolvido de modo a articular problema, solução, validação técnica e possibilidade de aplicação em contexto operacional.

A preparação da narrativa, o controle do tempo e a seleção adequada das evidências apresentadas contribuem para a clareza da exposição final.



Unidade 4

Integração Final e Defesa do Projeto

INSTITUIÇÃO EXECUTORA



COORDENADORA





Integração de Sistemas e Critérios de Qualidade de Entrega

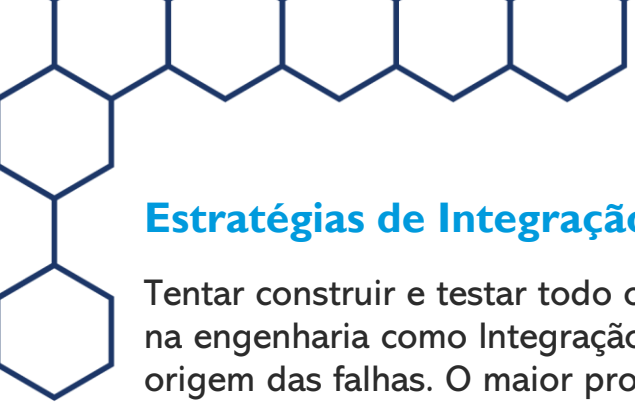
O Desafio da Integração Final

Existe uma interpretação recorrente no desenvolvimento de sistemas embarcados e soluções em IoT: a crença de que, se o hardware funciona na bancada e o código compila de forma isolada no simulador, basta conectar as duas coisas para que o sistema opere com sucesso. Na prática da engenharia, a etapa de integração costuma concentrar dificuldades relevantes do projeto.

A integração de sistemas não é uma simples soma aritmética de componentes. Ela envolve a interação entre diferentes elementos físicos e lógicos, como eletrônica, programação de firmware, redes de comunicação e modelos de inferência. Quando essas partes passam a operar em conjunto, podem surgir falhas que não aparecem durante os testes isolados.

São falhas que podem não aparecer quando os componentes são testados isoladamente. Um exemplo é um sensor analógico que realiza leituras estáveis e um modelo preditivo que classifica adequadamente os dados quando avaliados de forma independente. Durante a integração, porém, o acionamento de um motor na mesma rede elétrica pode introduzir ruído eletromagnético e gerar leituras inválidas no sensor, afetando os dados utilizados pelo algoritmo e aumentando a ocorrência de falsos positivos. Outro exemplo é um laço de repetição no firmware que funciona de forma estável isoladamente, mas que, ao interagir com uma rotina de reconexão do Wi-Fi, pode provocar um vazamento de memória (memory leak) e levar o microcontrolador a reiniciar após determinado período de operação.

A integração e os testes de sistema não devem ser tratados como um único evento de verificação ao final do projeto. Pelo contrário, trata-se de um processo contínuo e metódico de validação de interfaces. O grande desafio desta etapa final não é acrescentar novas funcionalidades, mas sim garantir que os subsistemas consigam coexistir de forma compatível, compartilhando recursos limitados de processamento e memória sem comprometer a estabilidade da arquitetura.



Estratégias de Integração: Evitando o "Big Bang"

Tentar construir e testar todo o sistema de uma única vez, uma prática conhecida na engenharia como Integração Big Bang, pode dificultar a identificação da origem das falhas. O maior problema dessa abordagem na fase final do projeto é a perda de rastreabilidade.

Se o sistema é ligado com todos os sensores, protocolos de rede e algoritmos ativados simultaneamente e ele trava, torna-se mais difícil identificar se o erro está no hardware, no firmware ou na infraestrutura de comunicação.

Para evitar esse cenário de baixa rastreabilidade próximo à entrega, a integração incremental tende a ser mais adequada. A integração pode ocorrer em passos menores, validando uma interface de cada vez.

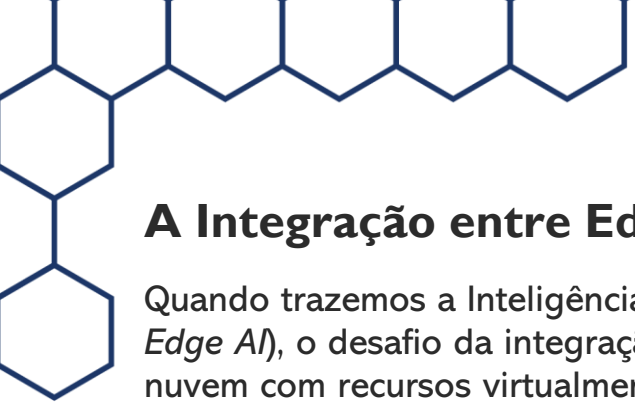
Aplicando esse conceito para o universo de IoT e *Edge AI*, a estratégia mais segura e eficiente é a **Integração Bottom-Up** (De Baixo para Cima). Em vez de conectar tudo de uma vez, a equipe deve construir e testar o sistema em camadas sucessivas:

Camada física, os sensores e microcontrolador: Primeiro, conecte apenas os sensores e atuadores na placa. Escreva um código simples, sem conexão Wi-Fi ou inteligência artificial, apenas para imprimir as leituras no monitor serial. O sensor está lendo corretamente? O botão aciona o relé? Se sim, essa camada está validada.

Camada de conectividade, a rede e protocolos: Adicione a rotina de conexão Wi-Fi e o *broker* MQTT ao código anterior. Teste o envio de *payloads* falsos ou das leituras recém-validadas. Os dados estão chegando na nuvem? A placa consegue manter a conexão estável?

Camada de aplicação, inteligência e visualização: Por fim, acople o modelo de inferência na borda ou conecte o *dashboard* (painel gerencial) na ponta da nuvem para consumir os dados do MQTT.

Uma vantagem da Integração *Bottom-Up* é o **isolamento de falhas**. Se, durante o Passo 3, o envio de dados parar de funcionar, a equipe sabe exatamente que o problema não está no sensor físico nem no hardware (pois o Passo 1 já havia sido validado). O erro está, indiscutivelmente, na forma como o algoritmo de inteligência artificial está interagindo com as rotinas de rede. A mitigação do risco pode ser mais direcionada.



A Integração entre Edge AI, Firmware e Hardware

Quando trazemos a Inteligência Artificial para a fronteira da rede (o chamado *Edge AI*), o desafio da integração se intensifica. Diferente de um servidor em nuvem com recursos virtualmente infinitos, o ecossistema de soluções industriais lida com uma escassez estrutural: pouca memória RAM, restrições de processamento e a necessidade de respostas em tempo real.

Em seu livro *TinyML*, Pete Warden detalha as restrições de rodar modelos de inferência, como algoritmos leves para visão computacional ou detecção de anomalias vibracionais, em microcontroladores.

A integração entre o modelo de machine learning, o firmware e o hardware físico exige coordenação arquitetural e lógica.

Considere o uso de plataformas robustas e muito utilizadas nesse escopo, como o ESP32. Esse microcontrolador possui um hardware capaz para aplicações de borda, gerenciando lógicas de atuadores e mantendo a pilha TCP/IP (como Wi-Fi e *brokers* MQTT) ativa. Contudo, quando a equipe decide embarcar o peso de uma matriz de inferência na mesma placa, o gargalo do processamento e do compartilhamento de memória se torna evidente.

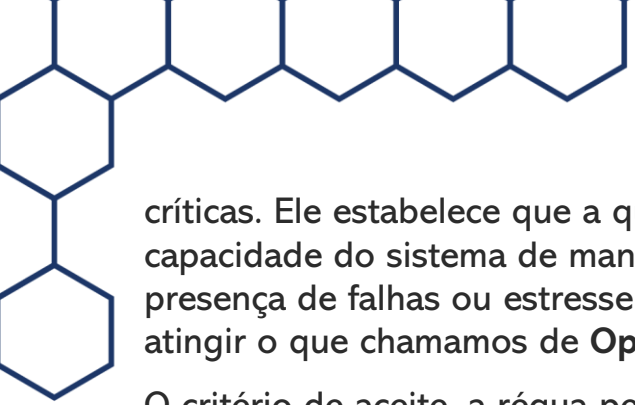
Um princípio importante da arquitetura em sistemas integrando hardware e software é: **o processamento da inteligência não pode bloquear a interface com o mundo físico.**

Se o laço principal do *firmware* passa 500 milissegundos travado calculando uma inferência sobre um vetor de dados, o que acontece se, exatamente nesse intervalo, uma peça atingir o fim de curso da esteira ou um operador acionar um botão de emergência? Se a arquitetura de software não utilizar interrupções de hardware (*hardware interrupts*) ou a divisão correta de núcleos para priorizar leituras críticas, o sistema ficará temporariamente sem resposta para determinados eventos. O modelo preditivo deve atuar em harmonia com os sensores, garantindo que a inteligência artificial agregue valor à solução sem prejudicar a responsividade fundamental ao ambiente de operação.

Estabilidade e Operacionalidade Mínima (Critérios de Aceite)

Para um sistema embarcado que será submetido a uma banca de avaliação técnica, funcionar uma única vez sob condições ideais de laboratório não é uma medida de sucesso. Na engenharia, chamamos o cenário em que nada dá errado de *Happy Path* (Caminho Feliz). Contudo, a indústria real opera fora do Caminho Feliz.

Em sua obra *Engenharia de Software*, Ian Sommerville explora profundamente os conceitos de Confiabilidade (*Reliability*) e Tolerância a Falhas em arquiteturas



críticas. Ele estabelece que a qualidade de uma entrega é medida pela capacidade do sistema de manter um nível aceitável de serviço mesmo na presença de falhas ou estresse ambiental. Para uma entrega final, o sistema deve atingir o que chamamos de **Operacionalidade Mínima**.

O critério de aceite, a régua pela qual o avaliador medirá o seu projeto é, portanto, a resiliência. A equipe deve realizar testes de estresse respondendo a perguntas fundamentais:

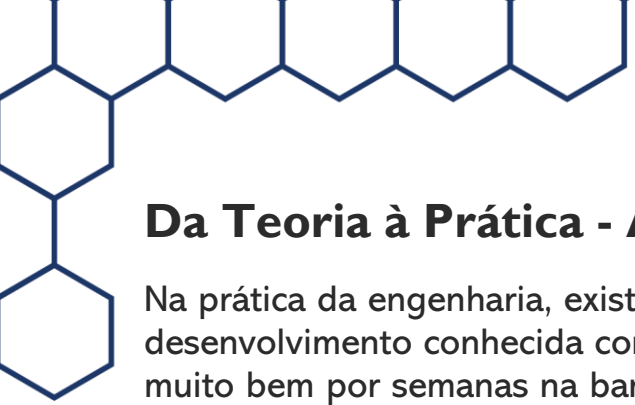
Conectividade: O que acontece se o roteador de internet for desligado? O seu *firmware* trava indefinidamente esperando resposta do protocolo MQTT, ou ele entra em modo de contingência local e se reconecta automaticamente assim que o Wi-Fi voltar?

Filtro de Ruído Físico: Se um operador esbarrar na máquina ou pressionar o botão várias vezes seguidas, os fios soltos e o ruído elétrico dos contatos causarão uma avalanche de leituras falsas? As técnicas de *debounce* (filtros por hardware ou software) estão implementadas para garantir que um pulso seja lido apenas uma vez?

Tratamento de Exceções: O que ocorre se o sensor desconectar e começar a enviar valores nulos ou fisicamente impossíveis (como temperatura de -999 °C)? O seu modelo de IA quebra e a placa congela?

Para garantir a estabilidade final, os desenvolvedores utilizam técnicas de programação defensiva. Isso inclui ferramentas estruturais do próprio microcontrolador, como o watchdog timer, um mecanismo de supervisão que reinicia a placa automaticamente caso perceba que o processador ficou preso em um loop infinito e parou de responder.

Um protótipo integrado de forma adequada é aquele que lida de forma autônoma com variações do ambiente, demonstrando condições de operação compatíveis com o contexto avaliado.



Da Teoria à Prática - A Ponte para a Defesa do Projeto

Na prática da engenharia, existe uma dificuldade recorrente entre equipes de desenvolvimento conhecida como o **Efeito Demonstração**: o sistema funciona muito bem por semanas na bancada do laboratório, mas entra em falha de execução, desconecta da rede ou apresenta dados corrompidos no exato momento em que o avaliador, o diretor ou o cliente pede para vê-lo funcionando.

Em muitos casos, o Efeito Demonstração não decorre de acaso, mas de fatores técnicos identificáveis, como integração final apressada, testes de fronteira insuficientes e ausência de critérios mínimos de operacionalidade.

Após a estruturação da apresentação executiva e da comunicação do valor da solução, a defesa do projeto também passa a expor a interação entre narrativa, implementação técnica e comportamento do sistema em demonstração. Nesse contexto, a avaliação não considera apenas a proposta da solução, mas também a capacidade da equipe de integrar hardware, firmware e inteligência artificial de forma estável e coerente com as condições de uso previstas.

O desempenho na entrega final reúne diferentes dimensões do projeto. Ele se evidencia quando a equipe consegue articular a clareza do Mapeamento de Valor, explicando o porquê do projeto existir, com um sistema integrando hardware e capaz de demonstrar, nas condições testadas e com baixa dependência de intervenção manual, a resposta proposta para a “dor”, ou necessidade industrial identificada.

A revisão das rotinas de reconexão, a verificação da sensibilidade a ruídos, a preparação de mecanismos de contingência e o planejamento da demonstração podem contribuir para uma apresentação final mais consistente.



Referências

Livros e artigos

BENDER, William N. *Aprendizagem baseada em projetos: educação diferenciada para o século XXI*. Porto Alegre: Penso, 2014.

BLANK, Steve; DORF, Bob. *The Startup Owner's Manual: The Step-by-Step Guide for Building a Great Company*. Hoboken: Wiley, 2020.

DUARTE, Nancy. *Slide:ology: The Art and Science of Creating Great Presentations*. Sebastopol: O'Reilly Media, 2008.

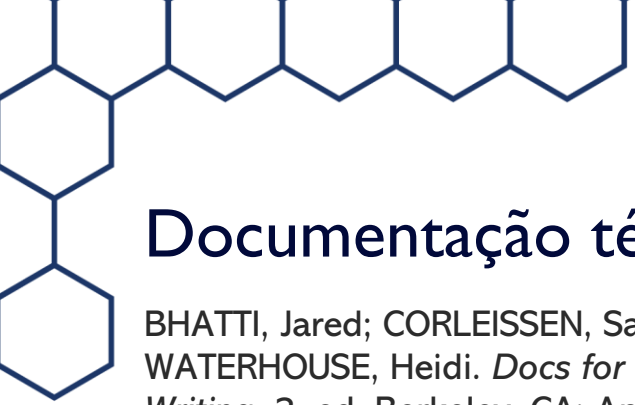
PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.

PROJECT MANAGEMENT INSTITUTE (PMI). *Um guia do conhecimento em gerenciamento de projetos (Guia PMBOK)*. 7. ed. Newtown Square: PMI, 2021.

RIES, Eric. *A startup enxuta: como os empreendedores atuais utilizam a inovação contínua para criar empresas extremamente bem-sucedidas*. São Paulo: Lua de Papel, 2012.

SCHWABER, Ken; SUTHERLAND, Jeff. *O Guia do Scrum: o guia definitivo para o Scrum: as regras do jogo*. Scrum.org, 2020.

WIEGERS, Karl; BEATTY, Joy. *Software Requirements*. 3. ed. Redmond: Microsoft Press, 2013



Documentação técnica e recursos on-line

BHATTI, Jared; CORLEISSEN, Sarah; LAMBOURNE, Jen; NUNEZ, David; WATERHOUSE, Heidi. *Docs for Developers: An Engineer's Field Guide to Technical Writing*. 2. ed. Berkeley, CA: Apress, 2026. Disponível em:

https://www.researchgate.net/publication/407097933_Docs_for_Developers_An_Engineer's_Field_Guide_to_Technical_Writing. Acesso em: 30 jul. 2026.

BLANK, Steve; DORF, Bob. *The Startup Owner's Manual: The Step-by-Step Guide for Building a Great Company*. Hoboken, NJ: John Wiley & Sons, 2020. Disponível em: <https://nashnw.myqnapcloud.com:8083/download/86/pdf/86.pdf>. Acesso em: 30 jul. 2026.

DUARTE, Nancy. *Resonate: Present Visual Stories that Transform Audiences*. Hoboken, NJ: John Wiley & Sons, 2010. Disponível em: <https://download.e-bookshelf.de/download/0000/5882/37/L-G-0000588237-0002385028.pdf>. Acesso em: 30 jul. 2026.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION (ISO); INTERNATIONAL ELECTROTECHNICAL COMMISSION (IEC); INSTITUTE OF ELECTRICAL AND ELECTRONICS ENGINEERS (IEEE). *ISO/IEC/IEEE 29148:2018: Systems and software engineering — Life cycle processes — Requirements engineering*. 2. ed. Geneva: ISO, 2018. Disponível em: <https://drkasbokar.com/wp-content/uploads/2024/09/29148-2018-ISOIECIEEE.pdf>. Acesso em: 30 jul. 2026.

PRESSMAN, Roger S. *Engenharia de software: uma abordagem profissional*. 7. ed. Porto Alegre: AMGH, 2011. Disponível em: [https://www.kufunda.net/publicdocs/Engenharia%20de%20Software%20-%207.ed.%20\(Roger%20S.%20Pressman\).pdf](https://www.kufunda.net/publicdocs/Engenharia%20de%20Software%20-%207.ed.%20(Roger%20S.%20Pressman).pdf). Acesso em: 30 jul. 2026.

RIES, Eric. *A startup enxuta: como os empreendedores atuais utilizam a inovação contínua para criar empresas extremamente bem-sucedidas*. São Paulo: Lua de Papel, 2012. Disponível em: <https://s-inova.ucdb.br/wp-content/uploads/biblioteca/a-startup-enxuta-eric-ries-livro-completo.pdf>. Acesso em: 30 jul. 2026.

WIEGERS, Karl; BEATTY, Joy. *Software Requirements*. 3. ed. Redmond, WA: Microsoft Press, 2013. Disponível em: [https://olivroqueaprende.com/WDK/Software Requirements 3rd Edition.pdf](https://olivroqueaprende.com/WDK/Software_Requirements_3rd_Edition.pdf). Acesso em: 30 jul. 2026.

PINAAT

INSTITUIÇÃO EXECUTORA

